# Problem Set 1

Due Friday Sep. 11, 10 am

## Comments

- This covers material in Units 1 and 2 as well as practice with Quarto.
- It's due at 10 am (Pacific) on September 11, both submitted as a PDF to Pensive as well as committed to your GitHub repository.

## Formatting requirements

1. Your electronic solution should be in the form of an Quarto file named `ps1.qmd`, with bash code chunks. Please see Lab 1, the Quarto and PS submission howtos on the course website, and the dynamic documents tutorial for more information on how to do this. If you want to initial work in a Jupyter notebook, that is fine, but you'll need to run `quarto convert file.ipynb` to generate a qmd file before rendering to PDF and submitting.
2. Using chunks of bash code in Qmd may be troublesome:

- Variables are not retained from bash chunk to bash chunk (i.e., state is not preserved between chunks), unlike with Python code chunks.
- We can help troubleshoot and feel free to post on Ed.
- **Test things out well before the due date with a dummy qmd file with a bash chunk to make sure things work.** We're quite happy to help in advance. We're not happy to help the night before the PS is due.

3. Your PDF submission to Pensive should be the PDF produced from your qmd. Your GitHub submission should include the qmd file and the final PDF, all named according to the submission guidelines.
4. Your solution should not just be shell code - you should have text describing how you approached the problem and what the various steps were. Your code should have comments indicating what each function or block of code does, and for any lines of code or code constructs that may be hard to understand, a comment indicating what that code does.
5. You do not need to (and should not) show exhaustive output, but in general you should show short examples of what your code does to demonstrate its functionality. Please see the grading rubric, and note that the output should be produced as a result of the code chunks being run during the rendering process, not by copy-pasting of output from running the code separately (and definitely not as screenshots).

6. Using `sed` in a basic way as shown in the bash tutorial might be useful. You should not need to use more advanced functionality nor should you need to use `awk`, but you may if you want to.

## Problems

1. Please read these lecture notes about how computers work, used in a class on statistical computing at CMU. Briefly (a few sentences) describe the difference between disk and memory based on that reference and/or other resources you find. If you're doing an data analysis, you should have an understanding of what situations might lead to running out of disk space and what situations might lead to running out of memory.

2. A friend of mine is planning to get married in Death Valley National Park in March (this problem is based on real events…). She wants to hold it as late in March as possible but without having a high chance of a very hot day. This problem will automate the task of generating information about what day of March to hold the wedding using data from the Global Historical Climatology Network. All of your operations should be done using the bash shell except part (c). Also, ALL of your work should be done using shell commands that you save in your solution file. So you can't say "I downloaded the data from such-and-such website" or "I unzipped the file"; you need to provide the bash code that someone else could run to repeat what you did. This is partly for practice in writing shell code and partly to enforce the idea that your work should be reproducible and documented.

    a. Download yearly climate data from the location above for a set of years of interest into a temporary directory. Do not download all the years and feel free to focus on a small number of years to reduce the amount of data you need to download (unzipped, some of the yearly files are as big as ~1 GB). Note that data for Death Valley is only present in the last few decades. As you are processing the files, report the number of observations in each year by printing the information to the screen (i.e., `stdout`), including if there are no observations for that year. Try to avoid printing anything else out (e.g., the progress of the downloading).

    b. Subset to the station corresponding to Death Valley, to the TMAX (maximum daily temperature) variable, and to March, and put all the data into a single file. In subsetting to Death Valley, get the information programmatically from the `ghcnd-stations.txt` file one level up on the website. Do NOT type (hard code) in the station ID code when you retrieve the Death Valley data from the yearly files.

    c. Write a small Python script (or R would be fine too) that takes as input your single file from (b) and makes a single plot showing side-by-side boxplots containing the maximum daily temperatures on each calendar day in March. Note that by creating a script and calling that from the shell you should avoid the challenges of mixing bash and Python chunks discussed in the Quarto references above.

    d. Now generalize your code from parts (a) and (b). Write a shell function whose arguments allow the user to specify (1) a string for identifying the location (this string should not be the station ID), (2) the weather variable of interest, and (3) the time period (i.e., the years of interest and the month of interest), and returns the results (and in this case don't print out the information about the number of rows). Your function should detect if the user provides the wrong number of arguments or a string that doesn't allow one to identify a single weather station and return a useful error message. It should also give useful help information if the

user invokes the function as: `get_weather -h`. Finally the function should remove the raw downloaded data files (alternatively, you should download such files into your operating system's temporary file location).